FANS — FACULDADE NOVA SERRANA
ENGENHARIA DE SOFTWARE
OTÁVIO DE PÁDUA RODRIGUES RIBEIRO

PROJETOFANS

Aplicativo colaborativo para registro de provas, entregas e material de aula por uma turma de graduação

NOVA SERRANA — MG
2026

OTÁVIO DE PÁDUA RODRIGUES RIBEIRO

PROJETOFANS

Aplicativo colaborativo para registro de provas, entregas e material de aula por uma turma de graduação

Trabalho apresentado às disciplinas Trabalho Interdisciplinar I e Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas, do 6º período do curso de Engenharia de Software da FANS — Faculdade Nova Serrana, como requisito de avaliação.

Professores: César Augusto Oliveira Soares e Daniel Lucas de Souza Lelis.

NOVA SERRANA — MG

2026

SUMÁRIO

1	**INTRODUÇÃO**	**4**
1.1	Problema	4
1.2	Justificativa	4
2	**OBJETIVOS**	**5**
2.1	Objetivo geral	5
2.2	Objetivos específicos	5
2.3	Delimitação do escopo	5
3	**PÚBLICO-ALVO**	**6**
4	**RECURSOS TECNOLÓGICOS UTILIZADOS**	**7**
4.1	Linguagens e frameworks	7
4.2	Banco de dados e versionamento do esquema	7
4.3	Ferramentas de produtividade e infraestrutura	7
5	**ARQUITETURA DA SOLUÇÃO**	**9**
5.1	Visão geral	9
5.2	Modelo de dados	9
6	**PRINCIPAIS FUNCIONALIDADES**	**11**
7	**NÍVEIS DE ACESSO E SEGURANÇA**	**12**
7.1	Níveis de acesso	12
7.2	Mecanismos de segurança	12
7.3	Auditoria de segurança e vulnerabilidades corrigidas	13
8	**DIFICULDADES ENCONTRADAS**	**14**
8.1	Indisponibilidade do ambiente de testes por vinte e sete dias	14
8.2	Conflito de versões do ambiente de execução Java	14
8.3	Falha inexistente provocada por cache de compilação	14
8.4	Demais dificuldades	15
9	**BENEFÍCIOS DA IMPLANTAÇÃO**	**16**
10	**CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS**	**17**
	REFERÊNCIAS	**18**

1 INTRODUÇÃO

Este documento descreve o *projetoFans*, um aplicativo para dispositivos móveis desenvolvido para que uma turma de graduação mantenha, de forma colaborativa, o registro do que precisa saber sobre o período letivo: as provas e entregas que se aproximam e o material apresentado em cada aula.

O sistema encontra-se em operação desde setembro de 2026, hospedado em servidor acessível pela internet, e vem sendo utilizado por integrantes da turma em seus próprios aparelhos. Não se trata, portanto, de protótipo ou de maquete de interface: o que este documento descreve está em funcionamento.

1.1 Problema

A informação acadêmica cotidiana de uma turma circula hoje, majoritariamente, por aplicativos de mensagem instantânea. Esse meio é adequado para *avisar*, mas inadequado para *consultar*. Um grupo de turma acumula centenas de mensagens por semana, de modo que a data de uma avaliação comunicada na terça-feira torna-se, na quinta-feira, praticamente irrecuperável sem releitura integral do histórico.

Decorrem desse formato três problemas recorrentes. O primeiro é o da submersão do aviso: a informação foi efetivamente transmitida, mas não pode ser localizada quando necessária. O segundo é o da irrecuperabilidade do material: o estudante ausente depende da disposição de um colega para obter o conteúdo da aula, o que transforma um direito acadêmico em favor pessoal. O terceiro é o da ausência de responsável: planilhas compartilhadas mantidas voluntariamente por um único estudante costumam ser abandonadas ao final de um ou dois períodos, pois dependem da persistência de uma só pessoa.

1.2 Justificativa

As plataformas institucionais de gestão acadêmica atendem à relação entre a instituição e o estudante — notas, frequência, matrícula — mas não ao registro horizontal que a turma produz entre si. Ambientes virtuais de aprendizagem, por sua vez, pressupõem adesão do docente: funcionam quando o professor alimenta a plataforma e deixam de funcionar quando ele não o faz.

O *projetoFans* ocupa deliberadamente o espaço intermediário. Seu funcionamento não depende de adesão institucional nem docente: qualquer integrante da turma registra informação, e o sistema opera integralmente ainda que nenhum professor jamais o acesse. A participação docente é possível e bem-vinda, mas permanece opcional por decisão de projeto.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver e implantar um aplicativo móvel que permita a uma turma de graduação registrar e consultar, de maneira colaborativa e sem necessidade de aprovação prévia, as avaliações e entregas futuras e o material didático de cada disciplina.

2.2 Objetivos específicos

- Apresentar, como tela inicial, as avaliações e entregas dos próximos quatorze dias, ordenadas por urgência e limitadas a seis itens, de modo a responder à pergunta “o que vem a seguir” em poucos segundos;
- Organizar o material didático por disciplina, distinguindo material de aula de material complementar, e vinculá-lo às avaliações correspondentes;
- Garantir que o estado pessoal de cada usuário — a marcação de um item como resolvido — permaneça isolado do registro compartilhado;
- Emitir aviso local no aparelho na véspera de cada avaliação, independentemente da disponibilidade do servidor no momento do disparo;
- Assegurar autoria visível e histórico de alterações em todo registro, de modo que a confiabilidade da informação possa ser avaliada por quem a consulta;
- Implantar a solução em ambiente acessível pela internet e validá-la com usuários reais em aparelhos próprios.

2.3 Delimitação do escopo

Delimitar o que não será construído é tão determinante quanto definir o que será. Foram explicitamente excluídos do escopo, por registro documentado no início do projeto: assinatura ou qualquer forma de monetização; geração automática de resumos por inteligência artificial; lançamento de notas ou boletim; integração com o sistema acadêmico da instituição; suporte a múltiplas turmas simultâneas por usuário; e envio e correção de trabalhos.

Cabe observação quanto ao penúltimo item. A estrutura do banco de dados comporta o vínculo de um usuário a diversas turmas, uma vez que o registro desse vínculo era necessário de todo modo para o ingresso por convite. A funcionalidade, contudo, não foi construída: a interface de programação resolve exatamente uma turma por usuário e não existe seletor de turma na interface gráfica. A distinção entre o que a estrutura comporta e o que o sistema oferece é deliberada e está documentada.

3 PÚBLICO-ALVO

O público-alvo primário é a turma de graduação presencial, compreendida como grupo de vinte a sessenta estudantes do mesmo período, cursando disciplinas comuns e já dotado de um canal de comunicação informal entre si.

Esse recorte não é acessório: é ele que sustenta a decisão de arquitetura mais sensível do sistema, a ausência de moderação prévia. Em um grupo cujos integrantes se conhecem pessoalmente, a autoria visível de cada registro constitui mecanismo de confiança suficiente, pois o custo reputacional de inserir informação falsa recai sobre pessoa identificável. Em um sistema aberto a público indeterminado, a mesma decisão seria inadequada e exigiria fila de aprovação.

O público-alvo secundário é o docente, na condição de contribuidor opcional. Sua conta exige aprovação administrativa, uma vez que o código de convite comprova pertencimento à turma, mas não comprova a condição de professor.

4 RECURSOS TECNOLÓGICOS UTILIZADOS

4.1 Linguagens e frameworks

O servidor foi implementado em Java 17 com o framework Spring Boot 3.5.3, organizado em camadas de controlador, serviço e repositório. Adotou-se a restrição de que entidades de persistência jamais atravessem a fronteira do controlador: toda resposta da interface de programação é construída a partir de objetos de transferência de dados montados explicitamente. Essa restrição impede que o resumo criptográfico da senha seja exposto por serialização automática — falha silenciosa e frequente em aplicações que devolvem entidades diretamente.

O aplicativo cliente foi construído com Ionic 8 sobre Angular 20, empregando componentes independentes e sinais reativos para gerência de estado. O empacotamento para Android utiliza Capacitor 8.5, que produz um arquivo instalável com acesso a recursos nativos, notadamente o agendamento de notificações locais.

A escolha de Java para o servidor considerou a maturidade do ecossistema e a verificação estática de tipos na camada responsável pela integridade dos dados compartilhados. A opção por Ionic e Angular permitiu manter uma única base de código para o aplicativo Android e para a execução em navegador.

4.2 Banco de dados e versionamento do esquema

Adotou-se o sistema gerenciador de banco de dados PostgreSQL 16. A decisão por banco relacional, em detrimento de alternativas orientadas a documentos, decorreu diretamente do modelo de dados: o vínculo entre avaliações e materiais é uma relação de muitos para muitos, e o histórico de alterações exige estrutura apensável e auditável. Ambas as características são atendidas naturalmente por banco relacional, ao passo que um banco orientado a documentos induziria à duplicação de dados e fragilizaria as regras de isolamento.

A evolução do esquema é versionada com Flyway, totalizando dez migrações numeradas. Observa-se a regra de que migração já aplicada nunca é editada: toda alteração dá origem à migração seguinte. O mapeamento objeto-relacional opera em modo de validação, verificando a correspondência entre o esquema esperado e o existente e interrompendo a inicialização em caso de divergência.

4.3 Ferramentas de produtividade e infraestrutura

O controle de versão é realizado com Git, em repositório hospedado no GitHub. A prototipação das interfaces foi conduzida em Figma, complementada por protótipos navegáveis em HTML mantidos no próprio repositório. O desenvolvimento contou com

programação assistida por inteligência artificial, mediante a ferramenta Claude Code, prática registrada de forma explícita: o repositório mantém diário de decisões com trinta e oito entradas datadas, cada qual identificando o que foi decidido, por qual razão e o que permaneceu pendente.

A hospedagem emprega serviços de camada gratuita: a interface de programação executa na plataforma Render e o banco de dados na plataforma Neon. Registre-se que a camada gratuita suspende a aplicação após quinze minutos de inatividade, com tempo de reativação que pode alcançar dois minutos. Essa limitação foi considerada no projeto, e não descoberta posteriormente: é precisamente por essa razão que o aviso da véspera é agendado localmente no aparelho, em vez de depender de processo em execução no servidor.

Quadro 1 — Síntese dos recursos tecnológicos

Camada	**Tecnologia**	**Versão e observação**
Linguagem do servidor	Java	17
Framework do servidor	Spring Boot	3.5.3
Automação de compilação	Maven	3.9 (por wrapper)
Banco de dados	PostgreSQL	16
Versionamento do esquema	Flyway	10 migrações
Framework do cliente	Ionic / Angular	8 / 20
Empacotamento móvel	Capacitor	8.5
Testes do servidor	JUnit 5 e PostgreSQL embutido	413 casos de teste
Testes do cliente	Karma e Jasmine	48 conjuntos
Hospedagem	Render e Neon	camada gratuita

Fonte: elaborado pelo autor (2026).

5 ARQUITETURA DA SOLUÇÃO

5.1 Visão geral

A arquitetura observa o princípio de que o cliente não decide regra de negócio. O aplicativo instalado no aparelho é superfície de apresentação dotada de cópia local para funcionamento sem conexão; a totalidade das regras reside no servidor.

Essa separação é mais que preferência estilística. A tela inicial deve exibir, no máximo, seis itens, restritos aos próximos quatorze dias e ordenados por urgência. Tais regras são computadas pelo servidor e entregues prontas ao cliente, que as renderiza na ordem recebida, sem reordenar, filtrar ou truncar. Caso fossem implementadas no cliente, cada tela poderia divergir ao longo do tempo sem que ninguém o percebesse — até que a turma deixasse de ser avisada de uma avaliação.

A comunicação entre cliente e servidor ocorre sobre HTTPS, autenticada por token JSON Web Token emitido pelo próprio servidor. Dezesseis controladores organizam os recursos: autenticação, turma, disciplina, material, evento, quadro pessoal, moderação, anotação, docente, administração e sincronização, entre outros.

O escopo de turma é resolvido por componente único, do qual todas as consultas dependem. Nenhum controlador filtra turma manualmente. Essa centralização cumpre duplo propósito: elimina a possibilidade de que uma rota nova omita a verificação por esquecimento e concentra em um só arquivo a alteração que seria necessária para suportar múltiplas instituições.

5.2 Modelo de dados

O modelo distingue rigorosamente *fato compartilhado* de *estado pessoal*. Avaliações, entregas e materiais constituem fatos compartilhados, visíveis a toda a turma. A marcação de um item como resolvido constitui estado pessoal e reside em tabela distinta, associada ao par usuário-evento.

A separação é estrutural, e não meramente convencional: não existe coluna de estado pessoal na tabela de eventos. Em consequência, a regra que proíbe o vazamento do estado de um usuário para os demais não depende da disciplina de quem programa — depende do banco de dados, que recusaria a operação. Há, inclusive, caso de teste que consulta o catálogo do próprio banco e falha caso tal coluna venha a ser criada.

Três outras decisões estruturais merecem registro. A exclusão é sempre lógica, mediante marcação temporal, pois a regra de preservação de histórico proíbe a remoção física de registros. Toda alteração grava, em tabela de revisões, o estado anterior do registro. E as chaves primárias são identificadores universais únicos, e não

sequências numéricas, o que evita enumeração e permitiria a consolidação futura de dados de instituições distintas sem colisão.

6 PRINCIPAIS FUNCIONALIDADES

O sistema organiza-se em quatro telas principais, cada qual associada a uma regra determinante.

Tela inicial. Apresenta as avaliações e entregas dos próximos quatorze dias, limitadas a seis itens e ordenadas por data crescente. A ordenação é por urgência, nunca por tipo: uma entrega marcada para o dia seguinte precede uma avaliação marcada para dali a quatro dias. O agrupamento por tipo é vedado. Quando não há itens no período, a tela exibe essa informação explicitamente, sem preenchê-la com sugestões ou conteúdo promocional — a ausência de compromissos é resposta correta e útil.

Disciplinas. Reúne, por disciplina, o material de aula e o material complementar, apresentados em seções distintas. Na tela de uma avaliação são exibidos exclusivamente os materiais a ela vinculados, e não o acervo integral da disciplina, de modo a responder objetivamente à questão “o que estudar para esta prova”.

Calendário. Oferece visão mensal navegável entre meses, com marcação por disciplina nos dias que contêm compromissos, e detalhamento do dia selecionado. Agrega também as anotações registradas pela turma.

Quadro pessoal. Organiza os itens do usuário em duas colunas, a fazer e feito. Conforme exposto na seção 5.2, a marcação não altera o registro compartilhado. Abaixo das colunas, seção recolhida reúne os registros do próprio usuário que tenham sido ocultados pela moderação.

Complementam o conjunto: ingresso em turma mediante código de convite de oito caracteres; cadastro de avaliações, entregas, disciplinas, materiais e anotações por qualquer integrante, sem aprovação prévia; notificação local na véspera de cada compromisso; e registro de autoria e de edição em todos os registros.

7 NÍVEIS DE ACESSO E SEGURANÇA

7.1 Níveis de acesso

O sistema define quatro papéis. Nenhum deles autoriza a edição de registro de autoria alheia.

Quadro 2 — Papéis e respectivas permissões

Papel	Permissões	Restrições
Aluno	Criar avaliações, entregas, disciplinas, materiais e anotações; editar e excluir exclusivamente os registros de sua autoria.	Não altera registro de terceiros; não acessa dados de outra turma.
Líder	Todas as permissões do aluno; ocultar da turma registro já publicado e solicitar ao autor sua exclusão.	Não edita conteúdo alheio nem impede a publicação. Altera visibilidade, jamais conteúdo.
Professor e Coordenador	Contribuir como os demais, com identificação de docente. O coordenador alcança as turmas sob sua coordenação.	Nenhuma permissão enquanto a conta não for aprovada administrativamente. O papel, isoladamente, não concede poder.
Administrador	Aprovar ou recusar contas docentes e vinculá-las a turmas.	Não acessa o conteúdo de turmas das quais não participe.

Fonte: elaborado pelo autor (2026).

A criação de conta é livre; o ingresso em turma exige o código de convite. Dessa separação decorre que o código de convite, e não o provedor de autenticação, constitui a fronteira de confiança do sistema.

7.2 Mecanismos de segurança

As senhas são armazenadas sob função de resumo criptográfico BCrypt, dotada de sal e fator de custo configurável, o que impede a recuperação do valor original. A autenticação por provedor externo é validada no servidor e convertida em token próprio, de modo que a interface de programação reconhece uma única forma de credencial.

O acesso a registro pertencente a outra turma resulta em resposta 404, e não 403. A escolha é deliberada: a resposta “proibido” confirmaria a existência do registro, de sorte que o próprio código de estado constituiria vazamento de informação.

A edição concorrente é tratada por controle otimista de versão. Toda alteração informa a versão lida; caso outro usuário tenha gravado no intervalo, a operação é recusada com resposta 409 e a interface recarrega o registro, em lugar de sobrescrever silenciosamente a edição alheia.

Soma-se a isso o limite de tentativas de autenticação e a invalidação das sessões ativas quando da alteração de senha — sem a qual a troca de senha posterior a um vazamento não removeria o acesso do invasor.

7.3 Auditoria de segurança e vulnerabilidades corrigidas

Foi conduzida auditoria de segurança sobre o próprio sistema, organizada por categorias de vulnerabilidade e documentada com indicação de arquivo e linha para cada constatação. Foram identificadas e corrigidas três falhas efetivamente exploráveis.

A primeira consistia em possibilidade de tomada de conta por meio da autenticação federada. O sistema empregava o endereço de correio eletrônico constante do token para identificar que dois acessos pertenciam à mesma pessoa, sem verificar se tal endereço havia sido confirmado pelo provedor. Dado que domínios corporativos podem emitir token com endereço não confirmado, quem o apresentasse assumiria a conta da vítima.

A segunda permitia a leitura de registro ocultado pela moderação mediante requisição direta por identificador. Como os identificadores já se encontravam nos aparelhos que haviam sincronizado, a moderação era contornável.

A terceira permitia que qualquer integrante desvinculasse materiais de avaliação alheia, operação destrutiva que não deixava rastro, pois o vínculo era removido fisicamente e o histórico não abrangia esse caso.

Cumpre destacar o método de verificação empregado. A aprovação de um caso de teste não demonstra que a proteção funciona; demonstra, apenas, que o teste não a exercita — proposições distintas. Para cada correção, portanto, a proteção foi *desativada propositalmente* e o teste reexecutado, confirmando-se a reprovação com a mensagem esperada. Somente então a proteção foi restabelecida. Tal procedimento, análogo ao teste de mutação, comprovou que as três falhas eram reais e que os testes correspondentes efetivamente as detectam.

A auditoria registra igualmente os limites não cobertos, examinados na seção 10.

8 DIFICULDADES ENCONTRADAS

As três dificuldades que mais consumiram tempo não decorreram de complexidade algorítmica, mas de ambiente de desenvolvimento e de método — categoria que, na prática profissional, responde pela maior parte dos atrasos.

8.1 Indisponibilidade do ambiente de testes por vinte e sete dias

Os testes automatizados do servidor executavam contra banco de dados provisionado em contêiner, o que pressupõe serviço de contêineres em funcionamento. Em 3 de agosto de 2026 tal serviço deixou de iniciar na estação de desenvolvimento e não voltou a funcionar.

A consequência foi severa. O critério de conclusão de cada tarefa do servidor era a aprovação dos testes; durante vinte e sete dias não houve como demonstrar a conclusão de tarefa alguma. Havia duas alternativas: prosseguir escrevendo código sem verificação, acumulando risco, ou substituir a infraestrutura de testes.

Optou-se pela substituição. O contêiner foi trocado por PostgreSQL embutido, solução que descarrega e executa binário real do banco a partir do próprio teste, dispensando qualquer serviço previamente instalado. Descartou-se deliberadamente a alternativa de empregar banco em memória de compatibilidade aproximada, ainda que mais leve: o esquema depende de restrições de verificação, de índice parcial condicionado e de consulta ao catálogo do próprio banco. Um banco apenas semelhante aprovaria operações que o PostgreSQL recusa e recusaria outras que ele aceita. Suíte que fornece resultado incorreto é pior que suíte que não executa.

8.2 Conflito de versões do ambiente de execução Java

O servidor compila com Java 17; o empacotamento Android exige versão 21 ou superior. Como as ferramentas de compilação utilizam a variável de ambiente, e não o executável do caminho de busca, apontá-la para uma versão inviabiliza a outra. O conflito manifestou-se nos dois sentidos, em semanas distintas, por meio de mensagens de erro que não indicavam a causa real.

A solução não foi de código, e sim documental: registrou-se qual versão cada comando exige, acompanhada da mensagem de erro exata que surge quando a configuração está incorreta. Problema de ambiente resolve-se por documentação.

8.3 Falha inexistente provocada por cache de compilação

Um teste da interface acusava reprovação em comportamento que o código-fonte, lido atentamente, tornava impossível. A instrumentação dos sinais reativos no

instante exato da verificação revelou divergência entre o código em disco e o código em execução: o cache de compilação servia versão anterior.

A funcionalidade estava correta e quase foi integralmente reescrita. A limpeza do cache restabeleceu a aprovação. O episódio passou a integrar a documentação do projeto como primeira verificação diante de falha aparentemente impossível — procedimento que custa quarenta segundos e evitou a repetição desnecessária de uma tarefa completa.

8.4 Demais dificuldades

Registram-se ainda: a limitação de cota da ferramenta de prototipação, que impediu a conclusão do protótipo e motivou a migração para protótipos navegáveis mantidos no repositório; e a suspensão automática da aplicação na camada gratuita de hospedagem, contornada por rotina periódica de requisição e, estruturalmente, pela decisão de agendar os avisos localmente no aparelho.

9 BENEFÍCIOS DA IMPLANTAÇÃO

Uma vez que o sistema se encontra implantado desde setembro de 2026, os benefícios a seguir não constituem expectativa, mas observação.

Para o estudante. A consulta substitui a busca: a data de uma avaliação ocupa posição fixa, em lugar de permanecer submersa no histórico de mensagens. O estudante ausente recupera o material da aula sem depender de solicitação pessoal. O aviso da véspera é emitido pelo próprio aparelho, funcionando ainda que o servidor esteja suspenso. E a pergunta sobre o que estudar para determinada avaliação encontra resposta objetiva nos materiais a ela vinculados.

Para a turma. A informação deixa de extinguir-se com o afastamento de quem a mantinha, pois a alimentação é distribuída entre todos os integrantes. A autoria visível permite avaliar a procedência de cada informação. Eventuais equívocos são corrigíveis, e a correção fica registrada.

Para o docente. Dispõe de canal para divulgação de material, com identificação de docente, sem que disso decorra obrigação. Trata-se de benefício aparentemente paradoxal que convém explicitar: ferramenta cuja operação dependa da adesão do professor deixa de funcionar quando um professor não adere. A opcionalidade é, portanto, condição de robustez.

10 CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS

O objetivo geral foi alcançado: o sistema encontra-se implantado, acessível pela internet e em uso por integrantes da turma em aparelhos próprios. Os objetivos específicos foram atendidos, com a ressalva relativa ao envio de arquivos, tratada adiante.

Entende-se que a contribuição mais relevante do projeto não reside em funcionalidade isolada, mas no rigor com que as regras invioláveis foram convertidas em estrutura verificável. A proibição de armazenar estado pessoal no registro compartilhado não constitui recomendação a ser observada por quem programa: constitui ausência de coluna, verificada por caso de teste que consulta o catálogo do banco.

Quanto aos limites conhecidos, registre-se, em observância ao princípio de que auditoria que omite limitações converte-se em propaganda: o isolamento entre turmas é assegurado exclusivamente na camada de aplicação, sem segunda barreira no próprio banco de dados; o limite de tentativas de autenticação existe, porém a restrição prática ainda é a capacidade do servidor em camada gratuita; não há política de segurança de conteúdo configurada no cliente, defesa em profundidade cuja ausência não corresponde a falha atual, mas reduz a margem diante de falha futura; e a plataforma iOS não foi submetida a teste, ainda que o código seja compatível.

Como trabalhos futuros, elencados por ordem de prioridade: implementar o envio efetivo de arquivos, hoje restrito a hiperligação, uma vez que a fotografia do quadro constitui provável caso de uso principal; implementar a notificação remota imediata quando da publicação de novo registro por colega; estabelecer a segunda barreira de isolamento no banco de dados; e publicar o aplicativo em loja de aplicativos, passo necessário à expansão para além da turma piloto.

Permanece aberta, por fim, questão de natureza não técnica que condiciona o investimento futuro: verificar junto à coordenação em que medida os sistemas institucionais já atendem ao registro do conteúdo ministrado por período. Caso o atendam satisfatoriamente, o sistema perde parte de sua razão de existir para esta turma — constatação que convém realizar antes, e não depois, de novo ciclo de desenvolvimento.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

GOOGLE. **Angular documentation**. [S. l.], 2026. Disponível em: https://angular.dev. Acesso em: 18 out. 2026.

IONIC. **Capacitor documentation**. [S. l.], 2026. Disponível em: https://capacitorjs.com/docs. Acesso em: 18 out. 2026.

IONIC. **Ionic Framework documentation**. [S. l.], 2026. Disponível em: https://ionicframework.com/docs. Acesso em: 18 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 documentation**. [S. l.], 2026. Disponível em: https://www.postgresql.org/docs/16/. Acesso em: 18 out. 2026.

RED GATE SOFTWARE. **Flyway documentation**. [S. l.], 2026. Disponível em: https://documentation.red-gate.com/flyway. Acesso em: 18 out. 2026.

VMWARE. **Spring Boot reference documentation**. [S. l.], 2026. Disponível em: https://docs.spring.io/spring-boot/docs/current/reference/html/. Acesso em: 18 out. 2026.